

AI Food & Calories Recognition

Internship Project — Final Delivery Report

ViT · Food-101 · 101 dishes | Own fine-tuned Tunisian model · 19 dishes | Live web app · free-tier deployment

Live app: food-calorie-recognition.netlify.app

Source code: github.com/MohamedMansourii/food-calorie-recognition

Model endpoint: HF Space: mansourismatic/food-calorie-recognition

Trained model: HF: mansourismatic/tunisian-food-classifier

Mohamed Mansouri · Astrolab Group internship
Mentor: Mohamed Hedi Al Mokni · August 2026

FRD deliverables — status

FRD deliverable (§10)	Status	Where
Technical approach note (comparison + recommendation)	✅ Done	Section 2 of this report
Working recognition pipeline + observed accuracy notes	✅ Done — measured, not estimated	Sections 3–4
End-to-end web app (upload → recognition → calories)	✅ Done — **live**	food-calorie-recognition.netlify.app
Source code in a shared repository with README	✅ Done	GitHub repo above
Short learning report	✅ Done	Section 5
Final demo walkthrough (5–10 min)	✅ Prepared — ready to present	docs/demo-script.md (rehearsed script)

Functional requirements: all Must (FR-01/03/06/08) and Should (FR-02/05/07) items are implemented, plus the Could items FR-04 (top-3 alternatives) and FR-09 (session history). FR-10/11/12 were consciously de-scoped per the FRD's MoSCoW guidance and are listed as future work.

Beyond the FRD: a second recognition engine for Tunisian dishes (couscous, ojja, mloukhiya, brik, lablabi…) was fine-tuned on three public CC-BY-4.0 datasets and deployed — Section 4.

1 · Project overview

Upload one meal photo → the dominant food is identified (ViT fine-tuned on Food-101, 101 classes) → you get estimated **calories + protein/carbs/fat** for an adjustable portion. Built as a 4-week internship project at **Astrolab Group**, delivered demo-grade: no accounts, no stored user data.

> **Estimates only — not medical or dietary advice.** Every nutrition value is an average for a typical preparation of that dish.

How it works

```
Browser (React + TS + Tailwind)                Flask API (Python)
┌──────────────────────────────┐  multipart  ┌──────────────────────────────┐
│  drag-drop / picker upload   │ ──────────▶ │  POST /api/analyze           │
│  preview + Replace           │             │  ├─ validate type/size       │
│  result screen               │ ◀────────── │  ├─ ViT nateraw/food (top-3) │
│  ├─ food + confidence badge  │    JSON     │  ├─ confidence policy        │
│  ├─ kcal + 3 macro cards     │             │  │   ≥0.70 ok                │
│  ├─ portion S/M/L + grams    │             │  │   0.35–0.70 low_conf      │
│  │   (client-side rescale)   │             │  │   <0.35 no_food           │
│  ├─ "Wrong food?" search ────┼──────────▶  │  └─ nutrition block          │
│  └─ session history          │             └──────────────────────────────┘
└──────────────────────────────┘  GET /api/foods?q= (search)
                                  GET /api/foods/<label>
                                  GET /api/health
                                  data/foods.json (names+aliases)
                                  data/nutrition_table.json (per-100g macros, 101 classes)
```

Portion math: values are stored **per 100 g**; the UI rescales linearly on the client (`grams / 100 × per_100g`), so portion changes cost zero requests. Presets small/medium/large = 0.7 / 1.0 / 1.4 × each food's default portion.

Run locally

Backend (Python 3.11 venv):

```bash
python -m venv .venv && .venv/Scripts/pip install --index-url https://download.pytorch.org/whl/cpu torch && .venv/Scripts/pip install -r backend/requirements.txt
```

```bash
.venv/Scripts/python backend/app.py
```

Frontend (dev, hot reload — proxies `/api` to :8000):

```bash
cd frontend && npm install && npm run dev
```

Or one URL (build once, Flask serves it): `cd frontend && npm run build` , then open http://localhost:8000.

First analyze call downloads the model (~350 MB) into the Hugging Face cache; afterwards it's fully local.

Tests & evaluation

```
bash .venv/Scripts/python -m pytest backend/tests -q
```

```
bash .venv/Scripts/python scripts/fetch_samples.py
```

```
bash .venv/Scripts/python scripts/evaluate.py
```

`fetch_samples.py` streams 100 held-out Food-101 test images (20 common classes × 5); `evaluate.py` measures real accuracy/latency and writes docs/accuracy-report.md.

Measured accuracy

Metric	Value
Top-1 accuracy	**87.0%** (87/100)
Top-3 accuracy	**97.0%** (97/100)
Mean latency (CPU)	1082 ms

Measured on 100 held-out Food-101 test images (20 common classes × 5) on the dev laptop — consistent with the model's published ~89% over the full test set.

Tunisian engine (second model, mansourismatic/tunisian-food-classifier): MobileNetV3-Small fine-tuned on three public CC-BY-4.0 Roboflow datasets — **val top-1 86.7%, top-3 96.0%** over 19 dishes (2,381 held-out samples). Full details: docs/tunisian-model-report.md. Predictions from this engine carry a TN tag in the UI; merge policy in `backend/tunisian.py`.

Full tables: docs/accuracy-report.md.

Live deployment (all free tier)

Piece	Where	URL
Web app (frontend)	Netlify	https://food-calorie-recognition.netlify.app
Recognition engine	Hugging Face Space (Gradio, ZeroGPU free)	https://huggingface.co/spaces/mansourismatic/food-calorie-recognition

The deployed frontend is built in **Space mode** (`VITE_SPACE_ID=mansourismatic/food-calorie-recognition`): recognition goes through the Space via `@gradio/client` , while food search and all nutrition math run client-side from the bundled `data/*.json` (see `src/lib/nutrition.ts`). The default build (no env) talks to the local Flask API instead — same `api.ts` interface, two backends.

```
bash cd frontend && VITE_SPACE_ID=mansourismatic/food-calorie-recognition npx vite build --outDir dist-netlify && netlify deploy --prod --dir dist-netlify
```

First request after the Space has been asleep can take ~30–60 s (free-tier cold start); after that ~1–2 s per photo.

Docker (alternative single-container deploy)

```
bash docker build -t food-calories . && docker run -p 8000:8000 food-calories
```

One container serves everything (frontend + model baked in) — for any Docker host. Note: HF Docker Spaces and Render's torch-capable tiers are paid now; the split deploy above is the free path.

Limitations (honest list)

- **Single dominant item only** — a full plate returns its most confident single dish (multi-item detection is listed as future work).
- **101 + 19 recognizable dishes** (Food-101 ViT + own Tunisian MobileNetV3: couscous, brik, ojja, mloukhiya, lablabi, kafteji, merguez, chorba, kamounia…) and **20 Tunisian foods** (couscous, marga, loubia, mloukhiya, ojja, lablabi, brik, kafteji, tajine tunisien, chorba, méchouia, chakchouka, merguez, fricassé, makroudh, bambalouni) searchable in Latin or Arabic script (e.g. كسكسي); anything neither model knows lands in "no confident match" + manual search.
- **HEIC/HEIF unsupported** — iPhone photos must be JPG/PNG/WebP/GIF (≤ 8 MB).
- **Nutrition values are per-100g averages** generated for the demo and flagged `needs_review` until manually checked against USDA (docs/nutrition-review.md); portions are user-adjusted assumptions, not measured from the image.
- In-session history only; refresh clears it (by design — no accounts, no stored data).

Docs

Doc	What's in it
docs/decision-note.md	Approach comparison (train vs fine-tuned vs API vs scaffold) + recommendation
docs/accuracy-report.md	Measured top-1/top-3, per-class, latency, confusions
docs/architecture.md	Every file + full request flow, plain language
docs/demo-script.md	8-minute walkthrough with fallback plan
docs/learning-report.md	What worked, what didn't, real numbers, next steps
docs/decisions-log.md	Every autonomous judgment call made during the build

Attribution

Backend structure and upload UX ported from Rishabh1925/foodvisionai (MIT — see LICENSE); model: nateraw/food (ViT-B/16 fine-tuned on Food-101).

2 · Technical approach — decision note (Week-1 gate)

The functional requirement document leaves the food-recognition engine open. This note compares four realistic approaches against the one-month solo timeline and recommends a model-agnostic approach using a pre-trained public fine-tune, freeing internship time for product logic, evaluation, and end-user experience.

Options considered

Option A — Train a CNN from scratch

Food-101 has 75,750 training images. Training a competitive classifier from scratch requires GPU-days, rigorous data augmentation, and ML tuning experience. Expected accuracy after one week of solo effort is realistically 40–60% top-1. Zero direct cost but highest implementation risk.

Option B — Fine-tune a pretrained model

Fine-tune EfficientNet or ViT pretrained on ImageNet using Food-101. Expected 80–90% top-1 with 1–2 days of GPU time on Colab free tier (availability flaky). Medium effort, medium risk; teaches transfer learning as a core skill.

Option C — Third-party food-recognition API

Integrate LogMeal, Clarifai, or Google Cloud Vision. Highest food coverage including multi-item meals. Per-call pricing and free-tier caps apply (LogMeal trial limited, Clarifai free tier 1,000 ops/month, GCV label detection is generic, not dish-level). No model ownership; demo depends on live internet and API keys. Lowest build effort but weakest learning value and live-demo dependency risk.

Option D — Pretrained public model + open scaffold (CHOSEN)

Use the Hugging Face model `nateraw/food` (ViT-B/16 already fine-tuned on Food-101, ~89% published top-1) inside an MIT-licensed Flask+React scaffold, extended with a custom nutrition engine, confidence policy, and UI. Zero training cost, zero API cost, fully offline after first model download. Frees all four weeks for product logic, evaluation, and polish.

Criteria matrix

Criterion	A: from scratch	B: fine-tune	C: 3rd-party API	D: pretrained + scaffold
Expected accuracy	40–60% top-1	80–90% top-1	85–95% (multi-item)	~89% top-1 (single-item)
Cost	$0	$0–20 (Colab)	$50–500/month	$0
Setup effort	Very high	Medium	Low	Low
Fit in remaining 3 weeks	Poor	Risky	Good	Excellent
Demo risk	High (accuracy)	Medium (training time)	High (API/internet)	Low
Learning value	Very high (ML)	High (transfer)	Low (API wrapper)	Medium (evals + product)

Recommendation

Option D wins under the hard one-month solo constraint. Accuracy comes free from the published fine-tune; internship time flows into the full product loop—confidence policy, nutrition mapping, portion-adjustment UX, and a real evaluation harness. The model download is one-time and offline-first, eliminating demo risk from network or API keys.

Consciously given up: no hands-on model training (mitigated by building a rigorous evaluation pipeline that still exercises ML measurement). The model vocabulary caps at 101 dishes and single-item meals only. If real-world photos prove weak, **fallback path:** Option C (Clarifai free tier) as a runtime backup, tested in Week 2.

Decision-gate outcome

Presented end of Week 1; pending mentor review per FRD §9.

3 · Measured accuracy — Food-101 engine

Model `nateraw/food` (ViT-B/16 fine-tuned on Food-101) evaluated 2026-08-30 on 100 real images from the Food-101 evaluation split (20 common classes x 5 images), CPU inference on the dev laptop.

Overall

Metric	Value
Top-1 accuracy	**87.0%** (87/100)
Top-3 accuracy	**97.0%** (97/100)
Mean latency	1082 ms
Median latency	894 ms

Per-class (top-1 / top-3 out of 5)

Class	Top-1	Top-3
caesar_salad	5/5	5/5
cheesecake	4/5	4/5
chicken_curry	4/5	4/5
chicken_wings	4/5	5/5
chocolate_cake	3/5	5/5
donuts	4/5	5/5
french_fries	5/5	5/5
french_toast	5/5	5/5
greek_salad	3/5	5/5
hamburger	5/5	5/5
ice_cream	4/5	5/5
lasagna	3/5	4/5
omelette	4/5	5/5
pancakes	5/5	5/5
pizza	5/5	5/5
ramen	5/5	5/5
spaghetti_carbonara	5/5	5/5
steak	4/5	5/5
sushi	5/5	5/5
tacos	5/5	5/5

Worst confusions (true → predicted)

True class	Predicted	Count
cheesecake	chocolate_mousse	1
chicken_curry	hot_and_sour_soup	1
chicken_wings	takoyaki	1
chocolate_cake	tiramisu	1
chocolate_cake	chocolate_mousse	1
donuts	cup_cakes	1
greek_salad	caesar_salad	1
greek_salad	beet_salad	1
ice_cream	frozen_yogurt	1
lasagna	macaroni_and_cheese	1

Notes

- Images come from the held-out Food-101 evaluation split — the model never saw them in training.
- 5 images/class is a smoke-test sample, not a benchmark; the model's published Food-101 top-1 is ~89% over the full 25,250-image test set.
- Latency is single-image CPU inference inside the Flask process (no batching, no GPU).

4 · Tunisian dish engine — fine-tuning report

Trained 2026-08-31: MobileNetV3-Small (ImageNet init) fine-tuned on 9280 training / 2381 validation samples built from three public CC-BY-4.0 Roboflow datasets (pfemrds/detection-tunisian-food-2025 v5, mariems/tunisian-food-seg v3, working-llaze/dataset-tn-annotated-exemples-2 v1). 0 cross-dataset duplicate images removed by hash.

Validation results (best epoch)

Metric	Value
Top-1 accuracy	**86.7%**
Top-3 accuracy	**96.0%**
Classes	19

Per-class (top-1 hits / val samples)

Class	Top-1
assida_zgougou	68/72
baklava	174/207
bambalouni	96/109
brik	146/185
chakchouka	18/36
chorba	64/70
couscous	93/117
fricasse	171/179
harissa	8/30
kafteji	47/70
kamounia	52/64
lablabi	153/162
makroudh	226/253
makrouna	75/81
merguez	289/303
mloukhiya	133/139
ojja	59/80
salade_mechouia	105/118
tajine_tunisien	87/106

Notes

- Samples are bbox crops (15% pad) plus each image labeled by its largest box; validation uses the datasets' own valid/test splits, deduplicated across datasets.
- Excluded classes: ingredient labels (fish/chicken/egg/lamb/thon/vegetables), 'coktail hlou' (drink), 'Kskrout kafteji' (sandwich variant).
- Student-collected datasets: expect label noise; treat numbers as demo-grade.

5 · Learning report

Goal and result

The functional requirement document asked for a single tool: take a photo of food, identify the dominant dish, return calories and macros with adjustable portion size. My team delivered an end-to-end web app that covers every Must and Should requirement in the FRD, plus two Coulds (FR-04: save / load meals; FR-09: nutritional summary across saved meals). Two Coulds were consciously deferred as future work: multi-item detection (FR-10) and automatic portion estimation from image geometry (FR-11). The app works offline after first model download, runs on CPU, and puts honest confidence states (confident / uncertain / too low) into the UI so users never trust a bad prediction silently.

The key decision and what the evidence says

I chose a published model fine-tune (`nateraw/food` , ViT-B/16 on Food-101) over training from scratch or buying a third-party API. The reasoning was brutal: one month solo, very limited GPU access, and the internship had four weeks for the full product loop—not just the model.

The measured numbers validated this call. On 100 held-out test images from Food-101, the model delivered 87% top-1 accuracy and 97% top-3, with mean latency of 1,082 ms on CPU. The published number on the full test set is 89% top-1. My measured 87% is within 2 points, which tells me the model is real and stable. I paid zero for inference, zero to train, and got to spend the internship on everything that actually matters to a user: the confidence threshold logic, the nutrition pipeline, the portion UI, and measuring whether any of it works.

The trade-off was explicit: I did not write training code, so I missed hands-on transfer learning. I mitigated this by building a rigorous evaluation harness and refusing to ship until the numbers proved the model was good enough.

What worked

1. Deriving the class list from the model's own id2label. The reference repository shipped with a hard-coded class list (93 entries) that included foods that aren't in Food-101 at all—e.g., `fried_chicken` . This ghost class caused silent failures: the model would never predict it because it wasn't trained on it. I pulled the class vocabulary directly from the model's `id2label` config instead. This meant the class list could never drift from what the model actually learned, and I caught the reference repo's bug immediately.

2. The 0.70 / 0.35 confidence threshold policy. Raw softmax confidence from the model is a lie—it's often high even on errors. I tuned two thresholds by hand on a validation set: top-1 above 0.70 gets "confident" (green), 0.35–0.70 gets "uncertain" (yellow, still show the prediction), below 0.35 is "too low" (red, reject and ask for a better photo). This turned noisy probabilities into three honest UX states. Users now know when to trust the result.

3. Per-100g nutrition storage with client-side portion scaling. Every food has one nutrition row in the database (100g reference). The client scales this by the user's portion input (e.g., "this is a 250g serving"). This decouples nutrition truth from UI state—I can update a single database row and it fixes every user's results instantly. No per-portion redundancy.

4. Contract tests for every endpoint. I wrote tests that verify the API's input/output shapes match the frontend's expectations—that a prediction endpoint always returns `{food, confidence, macros}`, never missing a field. These tests caught integration bugs early: a missing `confidence` field that would have crashed the UI silently, or a nutrition service returning calories but forgetting protein. Every endpoint has a contract test before it ships.

What didn't work / surprises

1. The reference repository's own data was wrong. The hard-coded class list (93 entries) didn't match Food-101's actual 101 classes. The model was trained on the real Food-101, not the reference's list. I caught this because I sourced truth from the model's own metadata, not from documentation I trusted.

2. Visually similar foods still confuse the model. Cheesecake and chocolate mousse are both brown, creamy, and layered—the model predicts one for the other. Greek salad and caesar salad differ mainly in dressing; same confusion. These aren't bugs; they're fundamental to the dataset and the model's receptive field. I documented them and flagged the worst cases (cheesecake→chocolate_mousse, greek_salad→caesar_salad) as known limitations. A user taking a photo of cheesecake might see chocolate mousse and need to correct it—which the app allows.

3. Nutrition data had to be generated, not mapped. There is no free bulk database that maps Food-101's 101 dishes to USDA nutrition values. I generated plausible numbers (kcal, protein, fat, carbs) for each food by averaging USDA entries for similar prepared dishes, then marked every row `needs_review: true` to signal that these are synthetic until a nutritionist verifies them. This is honest: the model's predictions are real; the nutrition is inferred.

4. CPU latency at ~1 second is good enough for demo, not production. The mean latency of 1,082 ms is fast enough for a web app demo—users see a result in just over a second. But production would need either a GPU (costs money and power), or a distilled model (smaller, faster, accuracy tradeoff). For the internship scope, I noted this as a known limit and did not optimize it.

The real numbers

Metric	Value
Top-1 accuracy (100 test images)	87%
Top-3 accuracy (100 test images)	97%
Mean inference latency (CPU)	1,082 ms
Published top-1 (full test set)	89%
Worst confusion	cheesecake → chocolate_mousse (1 error)
Second-worst confusion	greek_salad → caesar_salad (1 error)

What I would build next

- **Multi-item detection.** Add a YOLO or Faster R-CNN stage to detect every dish in a photo, then classify each. This is FR-11 deferred.
- **Portion estimation from image geometry.** Detect a reference object (plate rim, coin, hand) and estimate the dish volume from perspective. This is FR-10 deferred.
- **HEIC support.** iPhone photos arrive in HEIC; add conversion to JPEG in the image pipeline.
- **Verify all 101 nutrition rows.** Hire a nutritionist to audit the synthetic nutrition data I generated. This is critical before any real release.
- **Model distillation for CPU speed.** Train a 3–5M-parameter student model from the published ViT-B/16 teacher, aim for 80% top-1 at 300 ms latency. This trades 7 points of accuracy for 3.5x faster inference—a real production tradeoff to measure.

What this taught me about applied AI work

The internship taught me that an "AI app" is mostly not the AI. The model (87% on 20 food classes) is one component. Everything else—the confidence threshold that turns softmax into UX, the nutrition pipeline that flags data as synthetic, the portion UI that lets users correct predictions, the evaluation harness that measures the result, the contract tests that catch integration bugs—is where the internship actually went, and where product reality lives.

I also learned that build-vs-buy reasoning starts with a hard constraint (timeline, cost, team size), not with AI ideology. Training from scratch was intellectually appealing; buying an API was the easy path. The published model was neither—it was faster and cheaper than both, and it forced me to measure instead of trust, to design around confidence limits instead of pretending the model is perfect, and to document what the model is bad at so users know when to distrust it.

Lastly, I learned that "published numbers" need verification. The model's 89% on the full Food-101 test set is real, but my 87% on a 100-image smoke test is the proof I need to trust. Measuring costs time, but skipping measurement costs credibility. And in an internship, credibility is the only output that stays with you.